UNIVERSIDAD NACIONAL DEL ALTIPLANO
Facultad de Ingeniería Mecánica Eléctrica, Electrónica y Sistemas
Escuela Profesional de Ingeniería de Sistemas

GUÍA DE PRÁCTICA DE LABORATORIO

SEMANA 1

Revisión de POO I — Diagnóstico, Refactorización y Calidad de Código

Semestre 2026-II - Del 24 de agosto al 24 de diciembre de 2026

Curso:	Programación Orientada a Objetos II — SIS206
Semana:	1 (Diagnóstica)
Lenguaje:	Python 3.12+ / C++17
Duración:	2 horas de práctica (laboratorio)
Docente:	Mg. Aldo Hernán Zanabria Gálvez
ORCID / RENACYT:	0000-0003-3314-8768 | P0020608

Puno — Perú — 2026

I. OBJETIVO GENERAL

Diagnosticar y consolidar los conceptos fundamentales de Programación Orientada a Objetos adquiridos en SIS204, identificando las fortalezas y oportunidades de mejora de cada estudiante mediante la revisión, análisis y refactorización de código existente, estableciendo el punto de partida del Proyecto Integrador del semestre.

1.1 Objetivos Específicos

- Revisar y aplicar correctamente los cuatro conceptos core de POO: encapsulamiento, herencia, polimorfismo y abstracción.
- Identificar problemas de calidad en código existente: alta responsabilidad en una clase, bajo encapsulamiento, nombres inadecuados y acoplamiento innecesario.
- Aplicar técnicas básicas de refactorización: renombrado expresivo, extracción de métodos y encapsulamiento de atributos.
- Comprender los conceptos de cohesión y acoplamiento como métricas guía del diseño orientado a objetos.
- Iniciar la propuesta del Proyecto Integrador del semestre: definir el dominio, identificar los primeros actores y clases candidatas.

II. MARCO TEÓRICO

2.1 Los Cuatro Pilares de la POO — Revisión Rápida

La Programación Orientada a Objetos organiza el software en unidades llamadas objetos que combinan estado (atributos) y comportamiento (métodos). El paradigma reposa sobre cuatro pilares que este curso aplica de forma avanzada:

Pilar	Definición y aplicación en este curso
Encapsulamiento	Ocultar los detalles internos y exponer solo lo necesario. En este curso: invariantes, propiedades validadas, diseño por responsabilidades (Semana 2).
Herencia	Compartir e especializar comportamiento entre clases relacionadas. En este curso: jerarquías bien justificadas, principio de sustitución de Liskov (Semana 3).
Polimorfismo	Una misma interfaz, múltiples implementaciones. Dinámico (virtual/override) y estático (templates). Aplicado en colecciones heterogéneas (Semana 3-4).
Abstracción	Modelar solo lo relevante del dominio. Clases abstractas e interfaces como contratos (Semana 4). Base de los patrones de diseño (Unidad II).

2.2 Cohesión y Acoplamiento

La cohesión mide qué tan bien enfocada está una clase en UNA responsabilidad. El acoplamiento mide cuánto depende una clase de los detalles internos de otras. El objetivo de diseño es alta cohesión + bajo acoplamiento, que produce código más mantenible, extensible y testeable (Martin, 2008):

```
// BAJA cohesión — la clase hace demasiado (antipatrón "God Class") class
SistemaCompleto {     void registrarEstudiante() { ... }     void generarReporte()     {
... }     void enviarCorreo()       { ... }     void conectarBD()         { ... } };  //
ALTA cohesión — cada clase tiene UNA responsabilidad clara class RepositorioEstudiante {
void guardar(Estudiante e); }; class GeneradorReporte      { void
generar(vector<Estudiante>); }; class ServicioCorreo       { void enviar(string dest,
string msg); };
```

2.3 Refactorización: Mejorar sin Cambiar el Comportamiento

Refactorizar es transformar la estructura interna del código para mejorar su legibilidad, mantenibilidad y calidad, sin alterar su comportamiento observable desde el exterior (Fowler, 2018). Las técnicas más usadas en este curso son:

- Renombrado expresivo: variables, métodos y clases con nombres que revelan intención.
- Extracción de método: sacar un bloque de código repetido o complejo a un método con nombre descriptivo.
- Encapsulamiento de atributo: convertir atributos públicos en privados con getters/setters validados.
- Reemplazo de número mágico por constante con nombre semántico.



Relación con el Proyecto Integrador
Las decisiones de diseño que tome desde la Semana 1 (nombres de clases, responsabilidades, encapsulamiento) afectarán directamente la calidad de las semanas siguientes. Un buen diseño inicial hace que agregar herencia (Semana 3), colecciones (Semana 6) y patrones (Semanas 10-12) sea natural; un diseño apresurado lo convierte en un obstáculo.

⚠ **Prerequisito verificado antes del laboratorio**
Esta práctica asume que el estudiante domina: clases y objetos en C++ (struct vs class, constructores) y Python (def __init__, self), tipos de acceso (public/private/protected en C++, convención _priv en Python), y construcción básica de objetos en ambos lenguajes. Si hay dudas, revisar los materiales de SIS204 antes de iniciar.

III. MATERIALES Y HERRAMIENTAS

Herramienta	Uso
Python 3.12+ (Colab o local)	Implementación de actividades Python; diagnóstico inicial
g++ 10+ / MinGW-w64	Compilador C++17 — compilar con: g++ -std=c++17 -Wall -o prog archivo.cpp
Visual Studio Code / CLion	IDE principal del semestre; configurar desde la Semana 1
Git + GitHub	Crear el repositorio del proyecto integrador hoy mismo
Draw.io / StarUML	Primeros diagramas de clases del proyecto (Actividad 5)

IV. INSTRUCCIONES PREVIAS

1. Crear el notebook de Colab con el nombre: Practica_S1_POO2_ApellidoNombre.ipynb
2. Configurar el IDE y compilador C++17 en su equipo. Verificar: g++ --version (debe ser ≥ 10).
3. Crear HOY el repositorio de GitHub del Proyecto Integrador: POO2-2026II-[Apellido]. Subir un README.md inicial con el nombre del sistema candidato elegido.
4. Trabajar las Actividades 1-4 de forma individual. La Actividad 5 puede desarrollarse en equipos de 2-3 personas.

V. DESARROLLO DE ACTIVIDADES

ACTIVIDAD 1 — Diagnóstico — Identificación de problemas en código existente

Python + C++

Analice el siguiente código Python con problemas de diseño. NO lo ejecute aún — primero identifique todos los defectos de calidad que pueda encontrar.

```
# codigo_problematico.py  — identifique todos los problemas de diseño class est:     def
__init__(self, n, a, p):         self.n = n         self.a = a         self.p = p
self.h = []      def add(self, c, n):         self.h.append([c, n])      def ppa(self):
if len(self.h) == 0:             return 0         t = 0         for i in self.h:
t = t + i[1]         return t / len(self.h)      def pr(self):         print(self.n,
self.a, self.p, self.ppa())         for i in self.h:             print(i)  e1 =
est("Juan Mamani", 20210500, "Ing. Sistemas") e1.add("POO II", 16) e1.add("BD I", 14)
e1.pr()
```

Complete la tabla de análisis de calidad:

Defecto detectado	Línea(s)	¿Cómo lo corregirías?
Nombre de clase no descriptivo	1	

ACTIVIDAD 2 — Refactorización en Python — Misma lógica, mejor diseño

Python + C++

Reescriba el código de la Actividad 1 aplicando TODAS las correcciones identificadas. El resultado debe:

- Tener nombres descriptivos para la clase, atributos y métodos (snake_case para Python).
- Usar atributos privados (_nombre, _codigo) con propiedades @property para acceso controlado.
- Separar la lógica de negocio (calcular_ppa) de la presentación (mostrar).
- Reemplazar listas anidadas [c, n] por un dataclass Nota(curso: str, calificacion: float).
- Validar que la calificación esté en [0, 20] al agregar una nota.

```
# SU SOLUCIÓN REFACTORIZADA — Práctica S1, Actividad 2 from dataclasses import dataclass
from typing import Optional  @dataclass class Nota:     curso: str     calificacion:
float  class Estudiante:     def __init__(self, nombre: str, codigo: int, escuela: str):
# TODO: implementar con atributos privados y validación         ...      @property
def nombre(self) -> str:         # TODO: getter         ...      def agregar_nota(self,
curso: str, calificacion: float) -> None:         # TODO: validar [0, 20] y agregar Nota
...      def calcular_ppa(self) -> float:         # TODO: promedio de calificaciones,
0.0 si sin notas         ...      def __str__(self) -> str:         # TODO:
representación legible         ...  # Prueba — debe ejecutar sin errores: e =
Estudiante("Juan Carlos Mamani Quispe", 20210500, "Ing. Sistemas") e.agregar_nota("POO
II", 16) e.agregar_nota("Base de Datos I", 14) print(e) print(f"PPA:
{e.calcular_ppa():.2f}")  try:     e.agregar_nota("Redes", 25)    # debe lanzar
ValueError except ValueError as ex:     print(f"Error capturado correctamente: {ex}")
```

ACTIVIDAD 3 — Equivalente en C++17 — Encapsulamiento y validación

Python + C++

Implemente la clase Estudiante en C++17 con encapsulamiento estricto, equivalente a la Actividad 2. Compile con: g++ -std=c++17 -Wall -o estudiante main.cpp

```
// main.cpp — Práctica S1, Actividad 3 #include <string> #include <vector> #include
<stdexcept> #include <numeric> #include <iostream>  struct Nota {     std::string curso;
float       calificacion; };  class Estudiante {     std::string nombre_;     int
codigo_;     std::string escuela_;     std::vector<Nota> notas_;  public:
Estudiante(std::string nombre, int codigo, std::string escuela)         :
nombre_(std::move(nombre)), codigo_(codigo),           escuela_(std::move(escuela)) {}
const std::string& getNombre() const { return nombre_; }     int getCodigo() const {
return codigo_; }      void agregarNota(const std::string& curso, float cal) {
// TODO: validar 0 <= cal <= 20, lanzar std::invalid_argument si no
notas_.push_back({curso, cal});     }      float calcularPPA() const {         // TODO:
retornar promedio, 0.0f si sin notas         if (notas_.empty()) return 0.0f;
float suma = 0.0f;         for (const auto& n : notas_) suma += n.calificacion;
return suma / notas_.size();     }      void imprimir() const {         std::cout << "["
<< codigo_ << "] " << nombre_                    << " (" << escuela_ << ")"
<< "  PPA: " << calcularPPA() << '\n';     } };  int main() {     Estudiante e("Juan
Carlos Mamani Quispe", 20210500, "Ing. Sistemas");     e.agregarNota("POO II", 16);
e.agregarNota("Base de Datos I", 14);     e.imprimir();     try {
e.agregarNota("Redes", 25);     // debe lanzar     } catch (const std::invalid_argument&
ex) {         std::cout << "Error capturado: " << ex.what() << '\n';     }     return 0;
}
```

ACTIVIDAD 4 — Cohesión y acoplamiento — Análisis comparativo

Python + C++

Compare los dos diseños siguientes y responda las preguntas de análisis:

```
# DISEÑO A — todo en una clase class SistemaAcademico:     def
registrar_estudiante(self, nombre, codigo): ...     def agregar_nota(self, codigo_est,
curso, nota): ...     def calcular_ppa(self, codigo_est): ...     def exportar_csv(self,
filename): ...     def enviar_reporte(self, email): ...     def conectar_bd(self, host,
user, pw): ...  # DISEÑO B — responsabilidades separadas class Estudiante:     def
agregar_nota(self, nota: Nota): ...     def calcular_ppa(self) -> float: ...  class
RepositorioEstudiante:     def guardar(self, e: Estudiante): ...     def buscar(self,
codigo: int) -> Estudiante: ...  class ExportadorCSV:     def exportar(self,
estudiantes: list[Estudiante], archivo: str): ...
```

Pregunta de análisis	Su respuesta
¿Cuál diseño tiene mayor cohesión? ¿Por qué?	
¿Qué ocurre con el Diseño A si cambia el formato de exportación?	
¿Qué principio SOLID viola el Diseño A? (lo verán formalmente en la Semana 9)	
¿El Diseño B es más fácil de probar unitariamente? ¿Por qué?	

ACTIVIDAD 5 — Inicio del Proyecto Integrador — Propuesta y primeras clases

Python + C++

El Proyecto Integrador evolucionará semana a semana hasta la Semana 17. Hoy inicia con la propuesta del sistema candidato y el diseño inicial de las primeras clases.

- Elija UN sistema candidato del entorno regional o universitario. Ejemplos:
  Sistema de Registro Académico UNA-PUNO | Monitoreo de Sensores IoT Lago Titicaca | Gestión de Turismo del Altiplano | Inventario de Postas de Salud Rurales | Biblioteca Digital Regional
- Complete el siguiente formulario de propuesta:

Nombre del sistema: **Problema que resuelve:** **Usuarios principales:**	
3 clases candidatas iniciales:	
URL del repositorio GitHub:	https://github.com/

- Dibuje en Draw.io un diagrama de clases UML inicial con las 3 clases identificadas, mostrando sus atributos principales (con visibilidad), sin relaciones aún. Exporte como imagen PNG e incrústela en su notebook de Colab.

VI. PREGUNTAS DE REFLEXIÓN

- De todos los problemas identificados en la Actividad 1, ¿cuál considera el más perjudicial para el mantenimiento del código a largo plazo? Justifique.
- Explique con sus propias palabras la diferencia entre un atributo privado (__nombre en Python, nombre_ con acceso private en C++) y un atributo simplemente con nombre convencional (_nombre). ¿Por qué Python "confía" en el desarrollador mientras C++ "impone" la restricción?
- En la Actividad 4, el Diseño B tiene más archivos y clases. ¿Por qué, a pesar de eso, es preferible en un proyecto real que crece en el tiempo? ¿En qué contexto podría ser el Diseño A aceptable?
- ¿Por qué es importante crear el repositorio Git desde la Semana 1, aunque el código sea mínimo, en lugar de esperar a tener "algo que mostrar"?

VII. RÚBRICA DE EVALUACIÓN

Criterio	Excelente (4)	Bueno (3)	Regular (2)	Insuf. (1)	Peso
Análisis de defectos (Act. 1)	≥6 defectos correctamente identificados con corrección propuesta.	4-5 defectos identificados.	2-3 defectos.	< 2 o no entrega.	**20%**
Refactorización Python (Act. 2)	Código ejecutable, atributos privados, @property, validación y Nota dataclass.	Ejecutable con errores menores.	Parcialmente implementado.	No ejecuta.	**25%**
Implementación C++17 (Act. 3)	Compila sin warnings -Wall, validación correcta, output idéntico a Python.	Compila con warnings menores.	Compila pero falla la validación.	No compila.	**20%**
Análisis de diseño (Act. 4)	Respuestas técnicamente correctas con justificación propia.	Correctas pero sin profundidad.	Parcialmente correctas.	No responde.	**15%**
Proyecto integrador (Act. 5)	Sistema definido, 3 clases modeladas en UML, repo GitHub creado.	Sistema definido, sin diagrama UML.	Solo propuesta de sistema.	No entrega.	**20%**

VIII. REFERENCIAS

Fowler, M. (2018). Refactoring: Improving the Design of Existing Code (2nd ed.). Addison-Wesley. https://martinfowler.com/books/refactoring.html

Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.

Stroustrup, B. (2024). Programming: Principles and Practice Using C++ (3rd ed.). Addison-Wesley.

Python Software Foundation. (2026). Python 3.12 Documentation — Classes. https://docs.python.org/3/tutorial/classes.html

PARTE II

1. Alineamiento con el sílabo

Elemento	Definición
Contenido	Revisión de POO I: clases, objetos, atributos, métodos, constructores, control de acceso, cohesión y acoplamiento.
Actividad	Diagnóstico, revisión de código y refactorización de una solución previa.
Evidencia	Práctica diagnóstica, código refactorizado y justificación técnica.
Logro de unidad	Modela e implementa soluciones OO con criterios de cohesión, bajo acoplamiento y robustez.

2. Resultado de aprendizaje

Al finalizar · el estudiante diagnostica y refactoriza un módulo orientado a objetos, preserva su comportamiento mediante pruebas y demuestra mejoras en cohesión y acoplamiento con evidencia verificable.

3. Objetivos específicos

- Reconocer clase, objeto, estado, comportamiento, identidad y constructor en Python 3 y C++20.
- Identificar responsabilidades mezcladas, exposición indebida de estado y dependencias de infraestructura.
- Aplicar refactorización incremental sin alterar el comportamiento observable.
- Construir pruebas de caracterización antes de modificar el diseño.
- Comparar el diseño inicial y el refactorizado mediante métricas simples y una argumentación técnica.

4. Recursos y preparación

Recurso	Mínimo requerido	Verificación
Python	Python 3.11 o superior; pytest	python --version; pytest --version
C++	Compilador compatible con C++20	g++ --version
Editor	VS Code, CLion, PyCharm o equivalente	Compilar/ejecutar un archivo
Versionamiento	Git	git --version
Diagramación	PlantUML, diagrams.net o herramienta equivalente	Exportar PNG/PDF

Importante · el libro “Python para todos” explica correctamente los conceptos de clase, objeto, self, __init__ y encapsulación, pero sus ejemplos pertenecen a una versión antigua de Python. En esta práctica se utiliza Python 3; no copie la sintaxis print sin paréntesis ni clases heredadas de object por compatibilidad histórica.

5. Marco conceptual operativo

Concepto	Pregunta de control	Indicador de buen diseño
Clase	¿Qué responsabilidad representa?	Puede describirse sin usar varias veces “y”.
Objeto	¿Qué identidad y estado conserva?	Sus operaciones mantienen el estado válido.

Concepto	Pregunta de control	Indicador de buen diseño
Constructor	¿Qué debe cumplirse desde el nacimiento?	Rechaza datos intrínsecamente inválidos.
Cohesión	¿Sus métodos trabajan sobre una finalidad común?	Pocos motivos de cambio, vocabulario consistente.
Acoplamiento	¿Cuántos detalles externos conoce?	Dependencias explícitas, sustituibles y limitadas.
Encapsulación	¿Quién puede alterar el estado?	Operaciones con significado; colecciones protegidas.

6. Escenario: Gestor Académico Monolítico

La Escuela Profesional necesita registrar estudiantes, asignar notas, calcular promedios y generar un reporte simple. El prototipo heredado funciona en casos básicos, pero concentra reglas de dominio, persistencia, presentación y notificación en una sola clase. Cada nuevo formato de reporte obliga a modificar el mismo archivo y las listas internas pueden alterarse desde cualquier parte.

6.1 Reglas funcionales mínimas

- El estudiante tiene código, nombre y una colección de notas.
- Cada nota debe pertenecer al rango de 0 a 20.
- El promedio es 0 cuando todavía no existen notas.
- Un estudiante aprueba con promedio mayor o igual a 10.5.
- El reporte debe poder cambiar de texto a CSV sin modificar la entidad Estudiante.

7. Código base defectuoso — Python 3

Archivo: gestor_academico.py

```
class GestorAcademico:
    def __init__(self):
        self.estudiantes = []

    def registrar(self, codigo, nombre):
        self.estudiantes.append({"codigo": codigo, "nombre": nombre, "notas": []})

    def agregar_nota(self, codigo, nota):
        for e in self.estudiantes:
            if e["codigo"] == codigo:
                e["notas"].append(nota)
                with open("auditoria.txt", "a", encoding="utf-8") as f:
                    f.write(f"nota:{codigo}:{nota}\n")

    def reporte(self):
        salida = []
        for e in self.estudiantes:
            promedio = sum(e["notas"]) / len(e["notas"]) if e["notas"] else 0
            estado = "APROBADO" if promedio >= 10.5 else "DESAPROBADO"
            salida.append(f'{e["codigo"]};{e["nombre"]};{promedio:.2f};{estado}')
        with open("reporte.csv", "w", encoding="utf-8") as f:
            f.write("\n".join(salida))
        print("Reporte generado")
```

7.1 Diagnóstico obligatorio

Hallazgo	Ubicación	Riesgo	Refactorización candidata
Responsabilidades mezcladas	_____	_____	_____
Estado expuesto	_____	_____	_____
Validación ausente	_____	_____	_____
Dependencia concreta	_____	_____	_____
Duplicación potencial	_____	_____	_____

8. Código base defectuoso — C++20

Archivo: gestor_academico.cpp

```
struct Estudiante {
  std::string codigo;
  std::string nombre;
  std::vector<double> notas;
};

class GestorAcademico {
public:
  std::vector<Estudiante> estudiantes;
  void agregarNota(const std::string& codigo, double nota) {
    for (auto& e : estudiantes) {
      if (e.codigo == codigo) {
        e.notas.push_back(nota);
        std::ofstream log("auditoria.txt", std::ios::app);
        log << codigo << ";" << nota << "\n";
      }
    }
  }
};
```

9. Procedimiento de laboratorio

Fase	Tiempo	Acción	Evidencia
A. Línea base	15 min	Responder diagnóstico y ejecutar código base.	Captura de salida y commit baseline.
B. Caracterizar	20 min	Crear pruebas para los casos existentes y límites.	Pruebas en verde antes de refactorizar.
C. Modelar	20 min	Separar entidad, repositorio, reporte y auditoría.	Mapa de responsabilidades.
D. Refactorizar	45 min	Extraer clases e inyectar dependencias.	Código Python y C++.
E. Medir	20 min	Comparar dependencias, métodos y archivos afectados.	Matriz antes/después.
F. Defender	15 min	Explicar una decisión y un trade-off.	Video o sustentación breve.

10. Pruebas de caracterización mínimas

- Registrar un estudiante y recuperarlo por código.
- Agregar notas 0 y 20.
- Calcular promedio con tres notas conocidas.
- Devolver promedio 0 sin notas.
- Determinar aprobación en el límite 10.5.
- Rechazar nota -0.1 y 20.1.
- Rechazar código duplicado.
- Generar reporte sin alterar la entidad.

Referencia de prueba en pytest; adáptela a su diseño

```
def test_promedio_y_estado():
  e = Estudiante("E001", "Ana")
  e.registrar_nota(10)
  e.registrar_nota(11)
  assert e.promedio == 10.5
  assert e.esta_aprobado()

def test_rechaza_nota_fuera_de_rango():
  e = Estudiante("E001", "Ana")
  with pytest.raises(ValueError):
    e.registrar_nota(21)
```

11. Arquitectura objetivo sugerida

Componente	Responsabilidad	No debe conocer
Estudiante	Estado, notas, promedio y aprobación.	Archivos, consola, SQL.
RepositorioEstudiantes	Guardar y recuperar estudiantes.	Formato de reporte.
GeneradorReporte	Convertir datos a texto/CSV.	Cómo se persisten estudiantes.
Auditoría	Registrar eventos relevantes.	Reglas de promedio.
ServicioAcadémico	Coordinar el caso de uso.	Detalles de cada formato.

12. Mini experimento EXP-POO-001

Pregunta: ¿la separación de responsabilidades reduce el impacto de agregar un segundo formato de reporte?

Variable	Operacionalización
Tratamiento	Diseño inicial monolítico vs. diseño refactorizado.
Cambio solicitado	Agregar reporte JSON o HTML sin eliminar CSV.
Tiempo	Minutos desde lectura del cambio hasta pruebas en verde.
Impacto	Número de clases/archivos modificados.
Defectos	Pruebas que fallan por regresión.
Dependencias	Conteo manual de tipos externos conocidos por cada clase.

Indicador	Antes	Después	Interpretación
Clases/archivos afectados			
Dependencias de Gestor/Servicio			
Métodos de la clase principal			
Pruebas aprobadas / total			
Tiempo del cambio (min)			

13. Entregables

- Repositorio Git con commits baseline, tests, refactor y final.
- Código ejecutable en Python 3 y C++20.
- Pruebas automatizadas o, en C++, un ejecutable de pruebas reproducible.
- README con requisitos, comandos de ejecución y decisiones.
- Informe técnico de 2 a 4 páginas con diagnóstico, diseño, métricas y conclusión.
- Archivo CSV de métricas con identificador anonimizado del equipo.

14. Estructura del informe breve

1. Problema y línea base.
2. Olores de diseño identificados.
3. Responsabilidades propuestas.
4. Refactorizaciones aplicadas.
5. Resultados de pruebas y métricas.
6. Discusión: mejora lograda y costo introducido.
7. Declaración de uso de IA, si corresponde.
8. Referencias.

15. Rúbrica analítica — 20 puntos

Criterio	Destacado (4)	Logrado (3)	En proceso (2)	Inicial (1)	Peso
Diseño y responsabilidades	Separación coherente; bajo acoplamiento demostrado.	Separación adecuada con detalles menores.	Separación parcial; persisten mezclas.	Diseño monolítico o incoherente.	35%
Correctitud y pruebas	Todos los casos y fronteras; conducta preservada.	Casos principales y errores relevantes.	Pruebas incompletas o frágiles.	Sin evidencia reproducible.	25%
Calidad de código	Nombres, contratos y estructura consistentes en ambos lenguajes.	Código claro con problemas menores.	Duplicación o encapsulación débil.	Código difícil de ejecutar o explicar.	20%
Evidencia y argumentación	Métricas trazables; conclusión crítica y commits claros.	Evidencia suficiente y conclusión coherente.	Datos incompletos o interpretación superficial.	Sin medición ni justificación.	20%

16. Lista de cotejo antes de entregar

- [ ] El código base ejecuta antes de refactorizar.
- [ ] Las pruebas capturan conducta y valores frontera.
- [ ] La entidad no escribe archivos ni imprime reportes.
- [ ] Las dependencias externas se reciben o se aíslan.
- [ ] El repositorio muestra evolución mediante commits.
- [ ] Las métricas se calcularon con el mismo criterio antes y después.
- [ ] Todo integrante puede explicar el diseño.
- [ ] Se documentó el uso o no uso de IA.

17. Preguntas de análisis e investigación

- ¿Puede una clase tener pocos métodos y aun así baja cohesión? Fundamente.
- ¿Cuándo extraer una clase aumenta innecesariamente el acoplamiento?
- ¿Qué diferencia existe entre refactorización y reescritura?
- ¿Cómo cambiarían los resultados si el repositorio fuese una base de datos?
- Formule una hipótesis para comparar estudiante, IA y estudiante + IA en esta práctica.

18. Referencias

- González Duque, R. (s. f.). Python para todos. Capítulo “Orientación a objetos”, pp. 42–50. Material proporcionado para el curso.
- Fowler, M. (2018). Refactoring: Improving the Design of Existing Code (2nd ed.). Addison-Wesley. https://martinfowler.com/books/refactoring.html
- Martin, R. C. (2017). Clean Architecture. Pearson. https://www.pearson.com/en-us/subject-catalog/p/clean-architecture/P200000009528
- Python Software Foundation. (2026). Classes. https://docs.python.org/3/tutorial/classes.html
- cppreference.com. (2026). Classes. https://en.cppreference.com/w/cpp/language/classes
- pytest. (2026). Documentation. https://docs.pytest.org/